

Mid-Term Notes — Unit 1 & Unit 2

Bilingual (English + Hinglish) exam-focused notes with likely 2-mark and 5-mark questions.

EXAM PATTERN (ASSUMED) / PAPER PATTERN

Total = **20 marks** → 5 questions × 2 marks (10) + 2 questions × 5 marks (10)

Hinglish: Total 20 marks ka paper hai — 5 questions chhote (2-2 marks ke, definitions/truth tables/differences type) aur 2 questions bade (5-5 marks ke, diagram+derivation+explanation type).

Likely 2-mark topics	Likely 5-mark topics
Definitions, truth tables, short differences (Mux vs DMux, TTL vs CMOS, Register vs RAM, Latch vs Flip-flop)	Full derivations with diagram + HDL (AND/OR from NAND, Full Adder construction, DFF→Register→RAM building block chain, PC design)

UNIT 1 — Boolean & Arithmetic Foundations (Quick Revision)

Already covered in detail earlier — yahan sirf **exam-critical** points, taaki revise karna fast ho.

1.1 Functional Completeness of NAND

ENGLISH

NAND is functionally complete — every other gate (NOT, AND, OR, XOR) and eventually adders, memory, and a CPU can be built using only NAND gates.

HINGLISH

NAND se hi baaki sab gates ban sakte hain. NAND aur NOR hi duniya ke do aise gates hain jinme ye power hai.

$$\text{NOT}(a) = \text{NAND}(a, a)$$

$$\text{AND}(a, b) = \text{NOT}(\text{NAND}(a, b))$$

$$\text{OR}(a, b) = \text{NAND}(\text{NOT } a, \text{NOT } b)$$

1.2 Boolean Algebra — must-know laws

Law	Form
De Morgan's	$(a \cdot b)' = a' + b' \mid (a + b)' = a' \cdot b'$
Idempotent	$a \cdot a = a, a + a = a$
Complement	$a \cdot a' = 0, a + a' = 1$
Distributive	$a \cdot (b + c) = a \cdot b + a \cdot c$

1.3 Gates — formulas & truth tables (one-glance)

CORE GATE EQUATIONS

Gate	Formula
NOT	$\text{out} = a'$
AND	$\text{out} = a \cdot b$

OR	$\text{out} = a + b$
XOR	$\text{out} = a \cdot b' + a' \cdot b$
Mux	$\text{out} = a \cdot \text{sel}' + b \cdot \text{sel}$
DMux	$a = \text{in} \cdot \text{sel}', \quad b = \text{in} \cdot \text{sel}$

1.4 Half Adder & Full Adder

ENGLISH

Half Adder adds 2 bits (sum, carry). Full Adder adds 3 bits (a, b, carry-in) and is made from 2 Half Adders + 1 OR gate.

HINGLISH

Half Adder sirf 2 bits add karta hai. Full Adder 3 bits add karta hai (carry-in bhi) — do Half Adders aur ek OR gate jod ke banta hai.

sum = a XOR b

carry = a AND b

FullAdder: HalfAdder(a,b)→HalfAdder(sum1,c)→OR(carries)

1.5 Add16 (16-bit Ripple-Carry Adder)

16 Full Adders chained — carry ripples left to right. Final carry-out is **discarded** → addition is mod 2^{16} (overflow wraps silently).

1.6 TTL vs CMOS (one-glance)

Property	TTL	CMOS
Made of	BJTs	Complementary MOSFETs
Static power	High	Very low
Used today	Legacy	Almost all modern chips

Unit 1 — Probable Question Bank

What is functional completeness? 2M

EN: A gate type is functionally complete if every boolean function can be built from copies of it alone. NAND and NOR are the only single gates with this property.

HI: Ek hi gate (NAND) ki copies se baaki sab gates/functions ban sakna hi functional completeness hai.

Write the truth table of XOR gate. 2M

a=0,b=0→0 | a=0,b=1→1 | a=1,b=0→1 | a=1,b=1→0. Formula: $\text{out} = a \cdot b' + a' \cdot b$

Differentiate between Mux and DMux. 2M

EN: Mux selects one of two inputs to a single output based on sel. DMux routes one input to one of two outputs based on sel.

HI: Mux — do input mein se ek choose karke ek output deta hai. DMux — ek input ko do outputs mein se kisi ek mein bhejta hai.

What is the formula for carry in a Half Adder? 2M

carry = a AND b (= $a \cdot b$)

Differentiate TTL and CMOS logic families. 2M

EN: TTL uses bipolar transistors with high static power draw; CMOS uses complementary MOSFETs with very low static power, making it the standard for modern chips.

HI: TTL zyada power kharch karta hai (idle mein bhi), CMOS bahut kam power leta hai — isliye aaj sab chips CMOS ke hote hain.

Prove that AND(a,b) can be derived from NAND. Also construct a Full Adder using Half Adders, with diagram. 5M

EN: $\text{AND}(a,b) = \text{NOT}(\text{NAND}(a,b)) = \text{NOT}(\text{NOT}(a \cdot b)) = a \cdot b$ (double negation). Full Adder = HalfAdder(a,b) → gives sum1, carry1; HalfAdder(sum1,c) → gives final sum, carry2; $\text{carry} = \text{OR}(\text{carry1}, \text{carry2})$. Draw: two HA boxes feeding into an OR gate (see Unit 1 notes diagram).

HI: Pehle NAND ka proof likho (upar wala), fir Full Adder ka diagram banao — do Half Adder boxes, unke carries OR gate mein jaa rahe hain.

Explain how a 16-bit Add16 adder is built, and why the final carry is discarded. 5M

EN: 16 Full Adders (first stage is a Half Adder since no carry-in) are chained so each carry-out feeds the next stage's carry-in — this is "ripple carry". The 16th carry-out is unused/discarded, so the adder works modulo 2^{16} — overflow wraps around silently (two's complement behaviour).

HI: 16 Full Adders ek line mein jude hote hain, har wale ka carry agle ko milta hai. Last wala carry use nahi hota — isliye overflow hone pe answer wrap ho jaata hai, error nahi aata.

UNIT 2 — Sequential Logic & Memory Hierarchy

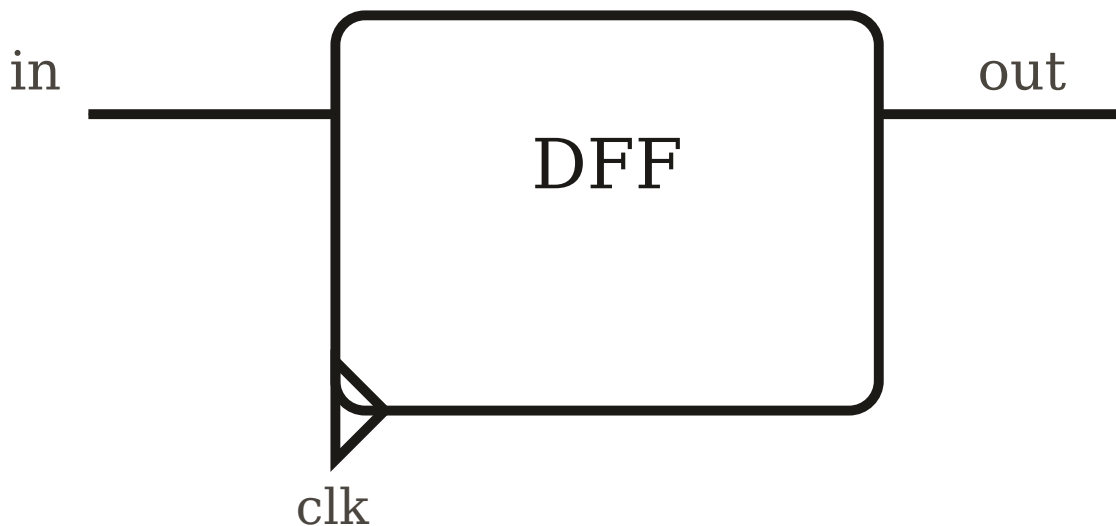
2.1 D Flip-Flop (DFF) & Binary Clocking Model

ENGLISH

Everything in Unit 1 was **combinational** — output depends only on current inputs, with no memory. Unit 2 introduces **sequential logic**: circuits that remember state over time. The basic memory element is the **D Flip-Flop (DFF)**: it samples its input `in` on each clock tick and outputs that value at the *next* time step — i.e. `out(t) = in(t-1)`.

HINGLISH

Unit 1 ke saare gates "combinational" the — matlab unka output sirf abhi ke input pe depend karta tha, koi memory nahi thi. Unit 2 mein "sequential logic" aata hai — jo apna purana state yaad rakhta hai. DFF sabse basic memory unit hai: ye har clock tick pe apna input store kar leta hai, aur agle step mein wahi value output karta hai.



DFF: stores input on clock edge, outputs it one cycle later

Exam-important: Clock has two phases — "tick" (rising edge) and "tock" (falling edge). All sequential chips in Nand2Tetris update synchronously on the clock, which is what makes multi-chip circuits (like RAM) behave predictably instead of racing.

DFF.hdl (built-in chip — no internal HDL, it's primitive)

```
CHIP DFF {
    IN in, clk;
    OUT out;
    // Primitive – behaviour: out(t) = in(t-1), sampled on clock edge
}
```

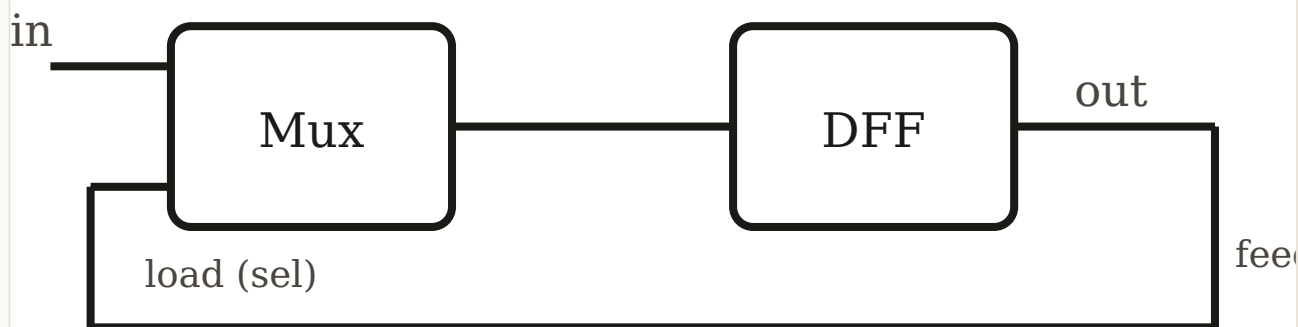
2.2 Bit, Register & Register File

ENGLISH

A plain DFF always loads a new value every clock tick. A **Bit** chip adds a `load` control: if `load=1`, store the new input; if `load=0`, keep the old value. This is built with a **Mux** feeding a DFF.

HINGLISH

Simple DFF har clock tick pe naya value store kar leta hai. "Bit" chip mein ek "load" control add hota hai — agar load=1 hai tabhi naya value store hoga, warna purana hi value wapas feed ho jaata hai (DFF ke output ko hi Mux se wapas input pe bhej dete hain).



Bit chip: $\text{Mux}(\text{in}, \text{feedback}, \text{load}) \rightarrow \text{DFF}$ — "remembers" when $\text{load}=0$

- **Register** = 16 Bit chips in parallel $\rightarrow$ stores one 16-bit value.
- **Register File / RAM** = many Registers, with addressing logic to pick which one to read/write.

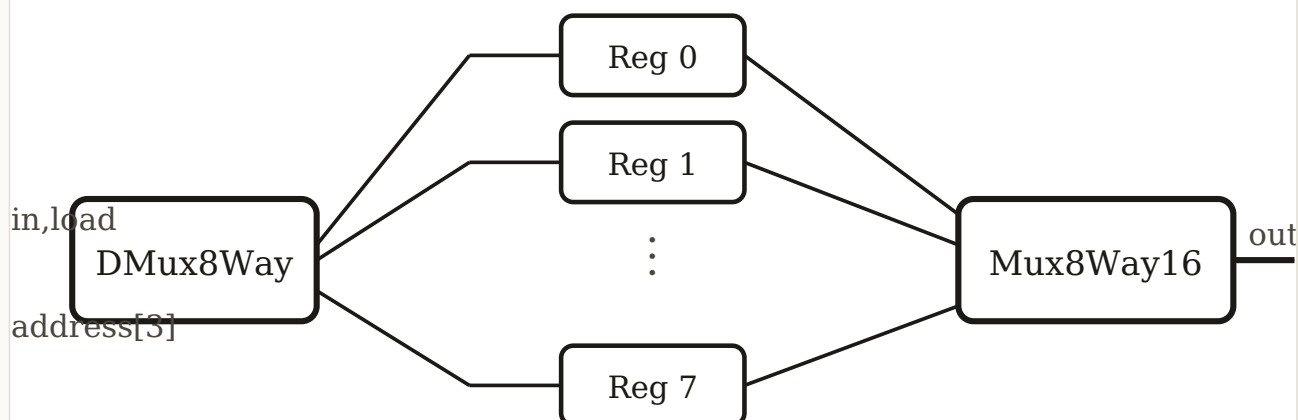
2.3 RAM Chips (8-register & 16K variants) — Addressing with DMux/Mux trees

ENGLISH

RAM is built hierarchically: RAM8 = 8 Registers, selected via a 3-bit address using a DMux8Way (to route the **load** signal to the correct register) and a Mux8Way16 (to select which register's output to read). Larger RAMs (RAM64, RAM512, RAM4K, RAM16K) are built the same way, by chaining 8 copies of the previous-size RAM chip.

HINGLISH

RAM chhote blocks se milkar banti hai. RAM8 mein 8 registers hote hain — 3-bit address se decide hota hai kaunsa register select karna hai. DMux8Way "load" signal ko sahi register tak bhejta hai, aur Mux8Way16 sahi register ka output wapas select karke deta hai. Bade RAM (RAM64, RAM4K, RAM16K) isi chhote RAM ki 8 copies jod ke banaye jaate hain — bas address ke extra bits use hote hain.



RAM8 structure: address picks which register to load (DMux) and which to output (Mux)

RAM SIZE VS ADDRESS BITS NEEDED

Chip	# Registers	Address bits

RAM8	8	3
RAM64	64	6
RAM512	512	9
RAM4K	4096	12
RAM16K	16384	14

Exam-important: Address bits = $\log_2(\text{number of registers})$. RAM16K needs 14 address bits because $2^{14} = 16384$.

2.4 Program Counter (PC)

ENGLISH

The PC is a 16-bit Register with extra control logic: it can **reset** to 0, **load** a new address, or **increment** by 1 — reset has highest priority, then load, then increment, else it holds its value.

HINGLISH

PC ek 16-bit register hi hai, bas usme 3 extra controls hote hain: reset (0 pe le jao), load (naya address daal do), inc (1 se badhao). Priority order: pehle reset check hota hai, fir load, fir inc — agar kuch bhi nahi to purana value hi rakhta hai.

PC.hdl (conceptual)

```
CHIP PC {
  IN in[16], load, inc, reset;
  OUT out[16];
  PARTS:
    Inc16(in=prevOut, out=incremented);
    Mux16(a=prevOut, b=incremented, sel=inc, out=afterInc);
    Mux16(a=afterInc, b=in, sel=load, out=afterLoad);
    Mux16(a=afterLoad, b=false, sel=reset, out=afterReset);
    Register(in=afterReset, load=true, out=out, out=prevOut);
}
```

2.5 Memory-Mapped I/O (Concept Preview)

ENGLISH

Input/output devices (keyboard, screen) are given reserved addresses within the same address space as RAM. Writing/reading those specific addresses doesn't access memory — it talks to the device. This is a *preview* concept; full detail comes in later units.

HINGLISH

Keyboard aur screen jaise devices ko bhi RAM ke address space mein hi kuch fixed addresses de diye jaate hain. Un addresses pe read/write karna memory access nahi, balki device se baat karna hota hai. Ye abhi sirf ek preview hai, detail aage ke units mein aayega.

2.6 Memory Hierarchy (Conceptual Link)

ENGLISH

The RAM you build here is the structural foundation for the broader memory hierarchy (cache → main memory → virtual memory/disk), which trades off speed vs size vs cost. Cache and virtual memory add extra management logic on top of this same basic RAM structure — covered in advanced/architecture courses.

HINGLISH

Jo RAM yahan banayi hai, wahi basic building block hai aage ki memory hierarchy (cache, main memory, virtual memory/disk) ka. Speed, size, aur cost ka trade-off hota hai — cache sabse fast par chhoti hoti hai, disk sabse slow par badi. Ye deep detail advanced courses mein milega.

2.7 Sequential Logic Applications

ENGLISH

Flip-flops are the base unit for: **Counters** (increment value every clock, used in PC), **Shift Registers** (shift bits left/right each clock, used in serial data transfer), and **State Machines** (circuit whose next state depends on current state + input — used in control logic/FSM design).

HINGLISH

Flip-flop se hi ye teeno bante hain: **Counter** (har clock pe value +1 hoti hai, PC isi ka example hai), **Shift Register** (bits ko left/right shift karta hai, serial data transfer mein use hota hai), **State Machine** (jiska agla state current state + input pe depend karta hai — control logic design mein kaam aata hai).

Unit 2 — Probable Question Bank

What is a DFF and how does it differ from combinational gates? 2M

EN: DFF is a memory element — it stores its input and outputs it one clock cycle later ($out(t)=in(t-1)$), unlike combinational gates whose output depends only on the present input.

HI: DFF memory rakhta hai — clock tick pe input store karke agle cycle mein output karta hai, jabki combinational gates ka output sirf abhi ke input pe depend karta hai.

How many address bits does RAM16K need, and why? 2M

14 bits, because $2^{14} = 16384 = 16K$ registers, so 14 bits are needed to uniquely address each one.

Differentiate between Register and RAM. 2M

EN: A Register stores a single 16-bit value; RAM is a collection of many registers accessed via an address, selecting which register to read/write.

HI: Register ek hi 16-bit value store karta hai; RAM bahut saare registers ka collection hai, jisme address se decide hota hai kaunsa register access karna hai.

What does the "load" input control in a Bit/Register chip? 2M

If $load=1$, the new input value is stored on the next clock tick; if $load=0$, the register keeps holding its previous value (via Mux feedback).

List the three control operations of a Program Counter (PC). 2M

reset (set to 0, highest priority), load (jump to given address), inc (increment by 1, lowest priority) — else hold value.

Explain, with a diagram, how a Register is built from DFFs, and how a RAM8 is built using Registers. 5M

EN: A single Bit = Mux(in, feedback-of-DFF-output, load) → DFF. A 16-bit Register = 16 such Bit chips in parallel, one per bit of the input. RAM8 = 8 Registers + DMux8Way (routes load signal using 3-bit address to the correct register) + Mux8Way16 (selects which register's 16-bit output to return). Draw: Bit diagram (§2.2) then the RAM8 tree diagram (§2.3).

HI: Pehle Bit chip ka diagram banao (Mux+DFF feedback loop), fir bolo Register = 16 Bits parallel mein. Phir RAM8 ka diagram banao — DMux8Way address ke hisaab se load bhejta hai, Mux8Way16 sahi register ka output nikal ke deta hai.

Explain the working of a Program Counter (PC) with its priority logic and HDL structure. 5M

EN: PC is a 16-bit Register with an Inc16 (incrementer) and chained Mux16s implementing priority: first check inc (increment if set), then load (overrides inc with new address), then reset (overrides everything, forces 0). Each clock tick, the resulting value is stored back via the Register. Write the PC.hdl structure (§2.4) and explain each Mux16 line.

HI: PC 16-bit Register hai jisme Inc16 (increment karne wala) aur chained Mux16 gates lagte hain jo priority set karte hain: pehle inc check hota hai, fir load (jo inc ko override kar deta hai), fir reset (sabse high priority, 0 force kar deta hai). Har clock tick pe final value wapas Register mein store ho jaati hai.

★ Most Likely to be Asked (Both Units Combined)

- **2M:** Functional completeness of NAND, XOR truth table, Mux vs DMux difference, TTL vs CMOS, DFF definition, RAM address-bit calculation, Register vs RAM difference, PC's three controls
- **5M:** AND/OR from NAND proof + Full Adder construction (diagram), Add16 + why final carry discarded, Register→RAM8 building chain (diagram), PC working with priority logic

Hinglish: Ye wahi topics hain jo exam mein sabse zyada chance rakhte hain — inko achhe se practice kar lo, especially diagrams haath se banana practice karo kyunki 5-mark wale questions mein diagram hi sabse zyada marks deta hai.